

Ramer–Douglas–Peucker algorithm

From Wikipedia, the free encyclopedia

(Redirected from Ramer-Douglas-Peucker algorithm)

The **Ramer–Douglas–Peucker algorithm**, also known as the **Douglas–Peucker algorithm** and **iterative end-point fit algorithm**, is an algorithm that decimates a curve composed of line segments to a similar curve with fewer points. It was one of the earliest successful algorithms developed for cartographic generalization. It produces the most accurate generalization, but it is also more time-consuming.[1]

Algorithm [edit]

The starting curve is an ordered set of points or lines and the distance dimension $\varepsilon > 0$.



Simplifying a piecewise linear curve with the Douglas–Peucker algorithm.

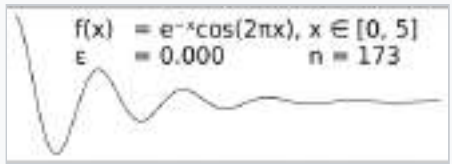
The algorithm recursively divides the line. Initially it is given all the points between the first and last point. It automatically marks the first and last point to be kept. It then finds the point that is farthest from the line segment with the first and last points as end points; this point is always farthest on the curve from the approximating line segment between the end points. If the point is closer than ε to the line segment, then any points not currently marked to be kept can be discarded without the simplified curve being worse than ε.

If the point farthest from the line segment is greater than ε from the approximation then that point must be kept. The algorithm recursively calls itself with the first point and the farthest point and then with the farthest point and the last point, which includes the farthest point being marked as kept.

When the recursion is completed a new output curve can be generated consisting of all and only those points that have been marked as kept.

Non-parametric Ramer–Douglas–Peucker [edit]

The choice of ε is usually user-defined. Like most line fitting, polygonal approximation or dominant point detection methods, it can be made non-parametric by using the error bound due to digitization and quantization as a termination condition.[2]



The effect of varying epsilon in a parametric implementation of RDP

Pseudocode [edit]

Assuming the input is a one-based array:

```
 # source: https://karthaus.nl/rdp/
function DouglasPeucker(PointList[], epsilon)
    # Find the point with the maximum distance
    dmax = 0
    index = 0
    end = length(PointList)
    for i = 2 to (end - 1) {
        d = perpendicularDistance(PointList[i], Line(PointList[1], PointList[end]))
        if (d > dmax) {
            index = i
            dmax = d
        }
    }

    ResultList[] = empty;

    # If max distance is greater than epsilon, recursively simplify
    if (dmax > epsilon) {
        # Recursive call
        recResults1[] = DouglasPeucker(PointList[1...index], epsilon)
        recResults2[] = DouglasPeucker(PointList[index...end], epsilon)

        # Build the result list
        ResultList[] = {recResults1[1...length(recResults1) - 1], recResults2[1...length(recResults2)]}
```

```
    } else {
        ResultList[] = {PointList[1], PointList[end]}
    }
    # Return the result
    return ResultList[]
```

Application [edit]

The algorithm is used for the processing of vector graphics and cartographic generalization. It is recognized as the one that delivers the best perceptual representations of the original lines. But a self-intersection could occur if the accepted approximation is not sufficiently fine which led to the development of variant algorithms.[3]

The algorithm is widely used in robotics[4] to perform simplification and denoising of range data acquired by a rotating range scanner; in this field it is known as the split-and-merge algorithm and is attributed to Duda and Hart.[5]

Complexity [edit]

The running time of this algorithm when run on a polyline consisting of $n-1$ segments and n vertices is given by the recurrence $T(n) = T(i+1) + T(n-i) + O(n)$ where $i = 1, 2, \ldots, n-2$ is the value of `index` in the pseudocode. In the worst case, $i = 1$ or $i = n-2$ at each recursive invocation yields a running time of $O(n^2)$. In the best case, $i = \frac{n}{2}$ or $i = \frac{n \pm 1}{2}$ at each recursive invocation yields a running time of $O(n \log n)$.

Using (fully or semi-) dynamic convex hull data structures, the simplification performed by the algorithm can be accomplished in $O(n \log n)$ time.[6]

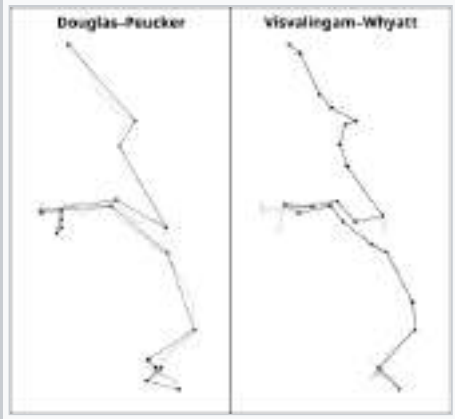
Given specific conditions related to the bounding metric, it is possible to decrease the computational complexity to a range between $O(n)$ and $O(2n)$ through the application of an iterative method.[7]

The running time for digital elevation model generalization using the three-dimensional variant of the algorithm is $O(n^3)$, but techniques have been developed to reduce the running time for larger data in practice.[8]

Similar algorithms [edit]

Alternative algorithms for line simplification include:

- Visvalingam–Whyatt
- Reumann–Witkam
- Opheim simplification
- Lang simplification
- Zhao–Saalfeld



Comparison with Visvalingam–Whyatt algorithm

See also [edit]

- Curve fitting

Further reading [edit]

- Ramer, Urs (1972). "An iterative procedure for the polygonal approximation of plane curves". *Computer Graphics and Image Processing*. **1** (3): 244–256. doi:10.1016/S0146-664X(72)80017-0.
- Douglas, David; Peucker, Thomas (1973). "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature". *Cartographica: The International Journal for Geographic Information and Geovisualization*. **10** (2): 112–122. doi:10.3138/FM57-6770-U75U-7727.
- Hershberger, John; Snoeyink, Jack (1992). *Speeding Up the Douglas–Peucker Line-Simplification Algorithm*. Proceedings of the 5th Symposium on Data Handling. pp. 134–143. UBC Tech Report TR-92-07 available at Speeding Up the Douglas-Peucker Line-Simplification Algorithm | Computer Science at UBC
- Duda, R.O.; Hart, P.E. (1973). *Pattern Classification and Scene Analysis*. New York: Wiley. Bibcode:1973pcsa.book.....D. Archived from the original on 2011-07-15.
- Visvalingam, M.; Whyatt, J.D. (1992). *Line Generalisation by Repeated Elimination of the Smallest Area* (Technical report). Discussion Paper. Cartographic Information Systems Research Group (CISRG), The University of Hull. 10.

References [edit]

1. ^ Shi, Wenzhong; Cheung, ChuiKwan (2006). "Performance Evaluation of Line Simplification Algorithms for Vector Generalization". *The Cartographic Journal*. **43** (1): 27–44. doi:10.1179/000870406x93490.
2. ^ Prasad, Dilip K.; Leung, Maylor K.H.; Quek, Chai; Cho, Siu-Yeung (2012). "A novel framework for making dominant point detection methods non-parametric". *Image and Vision Computing*. **30** (11): 843–859. doi:10.1016/j.imavis.2012.06.010.
3. ^ Wu, Shin-Ting; Marquez, Mercedes (2003). "A non-self-intersection Douglas-Peucker algorithm". *16th Brazilian Symposium on Computer Graphics and Image Processing (SIBGRAPI 2003)*. Sao Carlos, Brazil: IEEE. pp. 60–66. CiteSeerX 10.1.1.73.5773. doi:10.1109/SIBGRA.2003.1240992. ISBN 978-

0-7695-2032-2. S2CID 10163908.

4. ^ Nguyen, Viet; Gächter, Stefan; Martinelli, Agostino; Tomatis, Nicola; Siegwart, Roland (2007). "A comparison of line extraction algorithms using 2D range data for indoor mobile robotics" (PDF). *Autonomous Robots*. **23** (2): 97–111. doi:10.1007/s10514-007-9034-y. hdl:20.500.11850/9089. S2CID 35663952.
5. ^ Duda, Richard O.; Hart, Peter E. (1973). *Pattern classification and scene analysis*. New York: Wiley. ISBN 0-471-22361-1.
6. ^ Hershberger, John; Snoeyink, Jack (1992). *Speeding Up the Douglas-Peucker Line-Simplification Algorithm* (PDF) (Technical report).
7. ^ "ramer_douglas_peucker_funneling.py". *Gist*.
8. ^ Fei, Lifan; He, Jin (2009). "A three-dimensional Douglas–Peucker algorithm and its application to automated generalization of DEMs". *International Journal of Geographical Information Science*. **23** (6): 703–718. doi:10.1080/13658810701703001.

External links [edit]

- Boost.Geometry support Douglas–Peucker simplification algorithm
- Implementation of Ramer–Douglas–Peucker and many other simplification algorithms with open source licence in C++
- XSLT implementation of the algorithm for use with KML data.
- You can see the algorithm applied to a GPS log from a bike ride at the bottom of this page
- Interactive visualization of the algorithm
- Implementation in F#
- Ruby gem implementation
- JTS, Java Topology Suite, contains Java implementation of many algorithms, including the Douglas-Peucker algorithm
- Rosetta Code (Implementations in many languages)

Categories: Computer graphics algorithms | Geometric algorithms | Digital signal processing

This page was last edited on 29 November 2024, at 13:58 (UTC).

Text is available under the Creative Commons Attribution-ShareAlike 4.0 License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.

Privacy policy About Wikipedia Disclaimers Contact Wikipedia Code of Conduct Developers Statistics Cookie statement Mobile view

a WIKIMEDIA project

Powered by MediaWiki